



5



15



2250 sp

عملي

محتوى مجاني غير مخصص للبيع التجاري

كلية الهندسة المعلوماتية

السنة الرابعة

Variables and Functions in Shell Script

عملي مشترك



نظم التشغيل

RB Informatics; 2025/11/18

Shell Scripting

ما هو ال shell script ؟

ال shell script هو ملف نصي يحتوي على سلسلة من الأوامر التي ينفذها ال Shell .
بدلاً من كتابة الأوامر واحدة تلو الأخرى، نقوم بكتابتها داخل سكريبت وتشغيلها كبرنامج واحد.

لماذا نكتب Shell Scripts ؟

1. لتجنب التكرار : إذا كنت تقوم بتنفيذ سلسلة من الأوامر بشكل متكرر، لماذا لا نجعلها في أمر واحد؟
2. لتسهيل المهام المعقدة : بعض الأوامر تحتوي على خيارات معقدة يصعب تذكرها أو تطبيقها في كل مرة.

ومن أشهر أنواع ال Shells :

1. BASH (Bourne Again Shell) الأكثر استخداماً.
2. CSH (C Shell)
3. KSH (Korn Shell)

عناصر ال Script الأساسية:

لاحقة ملفات ال script : sh. ويمكن أن تختلف هذه اللاحقة حسب نوع ال shell

:Shebang/Header

نقوم هنا بتحديد نوع ال shell المستهدف (في حال لم نكتب نوع ال shell وكان ال script يوافق shell محددة أو إذا حددنا نوع ال shell ونفذنا ال script على نوع آخر فلن يعمل)

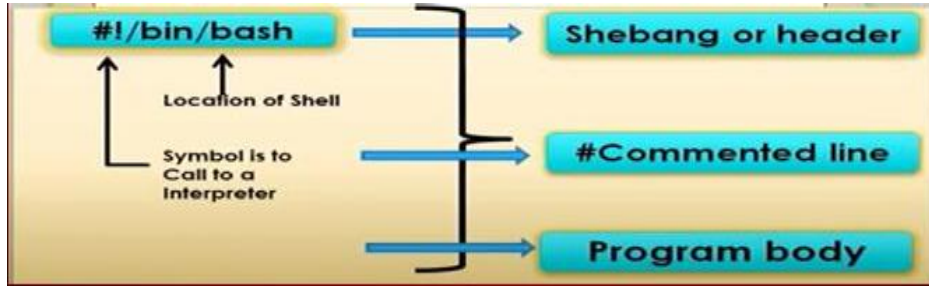
#!/bin/bash

#: يدل على توجيه المترجم (interpreter directive)

/bin/bash: المسار الكامل لل shell (هنا نوعها bash)

التعليقات:

أي سطر يبدأ بـ # يُعتبر تعليقاً ويُتجاهل عند التنفيذ.
تساعد على توضيح وظيفة السكريبت وتحسين قابلية القراءة.



كيفية تشغيل Shell Script

بشكل افتراضي، ملفات ال Scripts الجديدة لا تمتلك صلاحية التنفيذ لأسباب أمنية. لذلك لتشغيل السكريبت نقوم بإعطاء صلاحية التنفيذ أولاً: `chmod +x script-name.sh` ثم نقوم بتنفيذ السكريبت بالأمر: `./script-name.sh`.

المتغيرات في Shell Scripting

المتغيرات هي مواقع لتخزين القيم يمكن استخدامها أثناء تنفيذ السكريبت أو أثناء تنفيذ الأوامر مباشرة من ال terminal. للوصول إلى قيمة المتغير، نضع \$ قبل اسمه.

أنواع المتغيرات:

1. المتغيرات المعرفة من النظام (System Defined Variables - SDV):

- معرفة مسبقاً ومُدارة بواسطة نظام Linux
- تكون مكتوبة بحروف كبيرة دائماً.

2. المتغيرات المعرفة من المستخدم (User Defined Variables - UDV):

- ينشئها ويُديرها المستخدم.
- يُفضل كتابتها بحروف صغيرة (لكن يمكن كتابتها بأحرف كبيرة لن يعطي ذلك خطأ)

البحث عن الحقيقة

هي أول خطوة نحو المعرفة ...



اسم المتغير	مثال على القيمة	المعنى
BASH	/bin/bash	اسم الـ Shell
BASH_VERSION	5.0.17(1)-release	إصدار الـ Shell
COLUMNS	80	عدد الأعمدة (عدد المحارف التي يمكن كتابتها في سطر واحد) في الـ terminal
HOME	/home/mohammad	المجلد الرئيسي للمستخدم
LINES	25	عدد الأسطر في الـ terminal
LOGNAME	Mohammad	اسم تسجيل الدخول
OSTYPE	Linux	نوع نظام التشغيل
PATH	/user/bin:/sbin:/bin:/usr/sbin	يعيد المسارات التي من الممكن إيجاد الأوامر بداخلها, مثل أمر ls
PS1	[\u@\h \W]\\$	شكل موجه الأوامر (Prompt)
PWD	/home/mohammad	المجلد الحالي
SHELL	/bin/bash	مسار الـ Shell
USERNAME	mohammad	اسم المستخدم الحالي

يمكن عرض جميع المتغيرات وقيمتها باستخدام الأمر : `printenv`

🔗 مثال :

المتغيرات BASH و BASH_VERSION تعمل فقط مع الـ shell من نوع bash, و سنرى ذلك في مثال عملي:

1. المستخدم الحالي يستخدم Bash وعند طباعة قيم المتغيرات, نلاحظ القيم التالية:

```
rbcs@ -VMware-Virtual-Platform:~$ echo $SHELL
/bin/bash
rbcs@ -VMware-Virtual-Platform:~$ echo $BASH
/usr/bin/bash
rbcs@ -VMware-Virtual-Platform:~$ echo $BASH_VERSION
5.2.21(1)-release
```

2. سنقوم بإنشاء مستخدم جديد باستخدام shell مختلفة عن Bash :

```
sudo useradd -m testUser
```

```
sudo passwd testUser
```

التعليمة الثانية هي لتحديد كلمة السر للمستخدم، وإذا لم نقم بتحديدنا، فلن نستطيع تسجيل الدخول إلى المستخدم الجديد.

ال shell الافتراضي عند استخدام useradd هو /bin/sh

3. عند محاولة طباعة \$BASH و \$BASH_VERSION للمستخدم الجديد، نلاحظ أن هذه المتغيرات لا تعطي أي

قيمة لأن shell المستخدم ليس Bash

```
rbcs@ -VMware-Virtual-Platform:~$ su - testUser
Password:
$ echo $SHELL
/bin/sh
$ echo $BASH
$ echo $BASH_VERSION
$
```

المتغيرات الخاصة في Shell Scripting :

اسم المتغير	مثال على القيمة	المعنى
\$0	Myscript.sh	اسم السكريبت الحالي
\$n	arg1, arg2	الوسيطات الموضعية (positional arguments) n = 1,2,3,.....
\$#	3	عدد ال arguments الممررة للسكريبت
\$*	arg1 arg2 arg3	كل ال arguments كسلسلة واحدة
\$@	arg1 arg2 arg3	كل ال arguments كقيم منفصلة وكل قيمة بين علامات تنصيص
\$?	0	حالة خروج آخر أمر تم تنفيذه (Exit status)
\$\$	12345	رقم العملية PID لل shell الحالي
\$_	67890	رقم العملية PID لآخر أمر تم تشغيله في الخلفية

مثال تطبيقي لفهم المتغيرات الخاصة:

```
#!/bin/bash
echo "Name of script: $0"
echo "First argument: $1, Second argument: $2"
echo "Number of arguments: $#"
```

```
echo "-- Iterating over \@ --"
i=1
for arg in \@; do
    echo "Argument $i: $arg"
    ((i++))
done
echo "-----"
echo "-- Iterating over \"\$@\" --"
i=1
for arg in "\@"; do
    echo "Argument $i: $arg"
    ((i++))
done
echo "-----"
echo "-- Iterating over \$* --"
i=1
for arg in $*; do
    echo "Argument $i: $arg"
    ((i++))
done
echo "-----"
echo "-- Iterating over \"\$*\" --"
i=1
for arg in "$*"; do
    echo "Argument $i: $arg"
    ((i++))
done
echo "-----"
echo "Exit status of last command: $?"
sleep 30 &
echo "PID of last background job: $!"
echo "Script completed"
```

لدينا ال script التالي ضمن ملف argument.sh:

- في السطر الأول وضعنا ال header لتحديد نوع ال shell الذي سيُنفَّذ به السكريبت، وهو Bash في هذه الحالة.
- في السطر التالي نطبع اسم ملف السكريبت نفسه عن طريق المتغير \$0.
- بعد ذلك نقوم بطباعة الوسيطات (arguments) التي يستقبلها السكريبت. يمثل \$1 الوسيط الأول، و \$2 الوسيط الثاني، وهكذا.
- يمكن تمرير عدد كبير من الوسيطات دون مشكلة، حتى لو لم يتم استخدام جميعها داخل السكريبت.
- أما المتغير \$# فيعيد إجمالي عدد الوسيطات الممررة للسكريبت، سواء تم استخدامها أم لا.

شرح المتغيرات الخاصة المستخدمة في الحلقات التكرارية

تستخدم المتغيرات \@ و \$* للتعامل مع جميع الوسيطات، لكن الفرق بينهما يظهر عند استخدام علامات التنصيص:

	\$* - \@	"\$*"	"\$@"
الشرح مثال	تقوم باعتبار كل كلمة argument	تعامل كل ال arguments الممررة ك argument واحد	تقوم باعتبار كل مجموعة كلمات ضمن إشارتي argument تنصيص ك argument
I Love OS1	I Love OS1	I Love OS1	I Love OS1
"I Love" "OS1"	I Love OS1	I Love OS1	I Love OS1
"I Love" OS 1	I Love OS 1	I Love OS 1	I Love OS 1

ملاحظة حول escape :

الرمز \ يُستخدم لطباعة رموز خاصة كما هي دون أن يتم تفسيرها من قبل ال shell
في هذا السكريبت استخدمناه لطباعة الرمزين \$ و " على شكل: \\$ و \"

شرح الأسطر الأخيرة:

- يعيد المتغير \$? حالة خروج آخر أمر تم تنفيذه.
القيمة 0 تعني أن الأمر نُفذ بنجاح، بينما تشير أي قيمة غير 0 إلى وجود خطأ.
مثال: إذا نفذنا أمراً مثل ls test والمجلد غير موجود، فستكون القيمة عادة 2..
- أما المتغير \$! فيعيد رقم العملية (PID) لآخر مهمة تم تشغيلها في الخلفية.
في هذا السكريبت يتم إرسال الأمر sleep 30 للعمل في الخلفية باستخدام الرمز &، وبالتالي سيُعيد المتغير رقم عملية ال sleep
وإذا لم توجد عمليات تعمل في الخلفية ستكون قيمة المتغير فارغة.

نتائج تنفيذ ال script للدخل: "OS1" "I Love"

```
root@master:/home/mohammad/scripts# ./argument.sh "I love" "OS1"
Name of script: ./argument.sh
First argument: I love, Second argument: OS1
Number of arguments: 2
-----
-- Iterating over $@ --
Argument 1: I
Argument 2: love
Argument 3: OS1
-----
-- Iterating over "$@" --
Argument 1: I love
Argument 2: OS1
-----
-- Iterating over $* --
Argument 1: I
Argument 2: love
Argument 3: OS1
-----
-- Iterating over "$*" --
Argument 1: I love OS1
-----
Exit status of last command: 0
PID of last background job: 8526
Script completed
```

٤ مثال آخر لتوضيح كيفية التعامل مع المتغيرات المدخلة والعمليات الحسابية

في هذا المثال نستقبل من المستخدم رقمين، ثم نقوم بحساب: الجمع - الطرح - الضرب - القسمة بينهما.

```
#!/bin/bash
echo " please enter the first number: "
read a
echo " pelase enter the second number: "
read b
echo " a+b value is $(( $a+$b ))"
echo " a-b value is $(( $a-$b ))"
echo " a*b value is $(( $a*$b ))"
echo " a/b value is $(( $a/$b ))"
```

شرح النقاط الأساسية:

- نستخدم التعليمة read لقراءة المدخلات من المستخدم، ويتم تخزين القيمة في المتغير الذي يلي أمر القراءة مباشرة.
- بالنسبة لاستخدام الأقواس المزدوجة (()) في العمليات الحسابية: تُستخدم لتنفيذ عملية حسابية داخلية، ثم إسناد الناتج للمتغير، أي أننا نحتاج لقوسين :
 - قوس العملية الحسابية.
 - وقوس الإسناد للمتغير الخارجي.
- عند إدخال حرف بدل رقم، يتعامل الـ shell مع القيمة كأنها 0 في العمليات الحسابية .
- إذا أدخل المستخدم قيمة حرفية تساوي اسم متغير آخر، مثل: $a = 6$ ، $b = a$ ففي هذه الحالة يتم تفسير b على أنه يحتوي قيمة المتغير a وليس حرفاً، وبالتالي تصبح قيمة $b = 6$ ولن تُعامل كقيمة صفرية.
- كما يجب الانتباه إلى أن إجراء عملية القسمة على صفر يؤدي إلى ظهور خطأ (Error) في Bash

نتيجة الـ script في حال:

أحد المتغيرين حرف

```
rbcs@ -VMware-Virtual-Platform:~$ ./math.sh
please enter the first number:
e
please enter the second number:
9
a+b value is 9
a-b value is -9
a*b value is 0
a/b value is 0
```

القسمة على صفر

```
rbcs@ -VMware-Virtual-Platform:~$ ./math.sh
please enter the first number:
5
please enter the second number:
0
a+b value is 5
a-b value is 5
a*b value is 0
./math.sh: line 9: 5/0: division by 0 (error token is "0")
```

أحد المتغيرين له قيمة حرفية تساوي اسم المتغير الآخر

```
rbcs@ -VMware-Virtual-Platform:~$ ./math.sh
please enter the first number:
6
please enter the second number:
a
a+b value is 12
a-b value is 0
a*b value is 36
a/b value is 1
```

ملاحظة:

للتعامل مع الأعداد العشرية نقوم باستخدام bc كالتالي:

```
echo " a+b value is $(echo "$a+$b" | bc)"
```

ويمكن تخزين القيمة في متغير آخر:

```
sum = $(echo "$a+$b" | bc)
```

■ التتابع

التابع هو عبارة عن جزء صغير من الكود داخل السكريبت، يمكن إعادة استخدامه عدة مرات، عوضاً عن تكرار نفس الأوامر أكثر من مرة، نضعها داخل تابع ثم نستدعيه عند الحاجة.

■ لماذا نستخدم الدوال؟

- تقليل تكرار الكود.
- تسهيل الصيانة والتعديل على السكريبت.
- تحسين قراءة السكريبت وتنظيمه.
- إمكانية تمرير arguments لزيادة المرونة في الاستخدام .

يمكن تعريف الدوال بطريقتين:

1. `Function_name () { <commands> }`
2. `function Function_name { <commands> }`

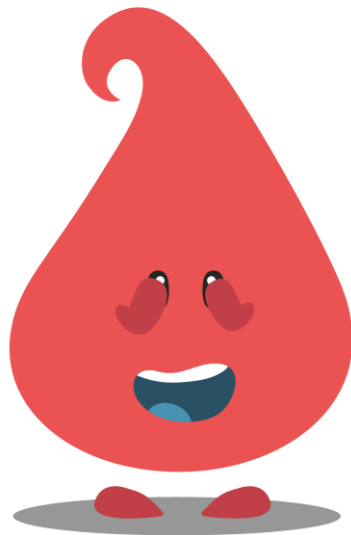
➤ أمثلة لتوضيح كيفية استخدام التتابع:

1. مثال يقوم بقراءة الاسم الأول و الأخير من دخل المستخدم و نستخدم تابع name لطباعته:

```
#!/bin/bash
echo "enter your first name: "
read a
echo "enter your second name: "
read b

name()
{
echo "you name is; $a $b"
}

name
```



2. قمنا بكتابة تابع يقبل argument ويقوم بطباعة رسالة ترحيبية:

```
#!/bin/bash

say_hello()
{
    echo "Hello $1, welcome to Bash functions!"
}

say_hello "User"
```

```
rbcs@ -VMware-Virtual-Platform:~$ ./hello_function.sh
Hello User, welcome to Bash functions!
```

ملاحظات هامة:

- لا يمكن استدعاء التابع قبل تعريفه، لأن الـ Shell يقرأ السكريبت وينفذه سطرًا بسطر، لذلك إذا حاولنا استدعاء التابع قبل كتابته، سيظهر خطأ يشير إلى أن التابع غير موجود.

```
#!/bin/bash

say_hello "User"

say_hello()
{
    echo "Hello $1, welcome to Bash functions!"
}

~
~
```

```
rbcs@ -VMware-Virtual-Platform:~$ ./hello_function.sh
./hello_function.sh: line 3: say_hello: command not found
```

- الرمز \$1 يمثل أول وسيط (argument) يتم تمريره للتابع. على سبيل المثال، إذا مررنا وسيطين إلى التابع، فإن: \$1 يمثل الوسيط الأول، \$2 يمثل الوسيط الثاني وبالتالي، إذا استخدمنا \$2 داخل التابع، سيتم طباعة الوسيط الثاني تحديدًا.

```
#!/bin/bash

say_hello()
{
    echo "Hello $2, welcome to Bash functions!"
}

say_hello "User1" "User2"

~
~
```

```
rbcs@ -VMware-Virtual-Platform:~$ ./hello_function.sh
Hello User2, welcome to Bash functions!
```

GLOBAL VS LOCAL VARIABLES IN BASH

■ المتغيرات العالمية (Global Variables)

بشكل افتراضي، أي متغير تقوم بإنشائه داخل سكربت Bash يكون Global. أي يمكن الوصول إليه والتعديل عليه من أي جزء داخل السكربت، بما في ذلك من داخل التتابع .

■ المتغيرات المحلية (Local Variables)

لجعل المتغير مرئياً داخل تابع فقط، نستخدم الكلمة المفتاحية local عند تعريفه لأول مرة. هذا يعني أن المتغير سيبقى محصوراً داخل نطاق التابع ولن يؤثر على أي متغير يحمل نفس الاسم خارج التابع.

```
local var_name=var_value
```

■ لماذا نستخدم المتغيرات ال local؟

- يمنع تعديل المتغيرات العالمية عن طريق الخطأ.
- يجعل التتابع أكثر استقلالية وأماناً.
- يساعد على تنظيم الكود وتحسين قابلية الصيانة.

➤ مثال:



```
#!/bin/bash
var_change (){
local var1='local1'
echo Inside function: var1 is $var1 : var2 is $var2
var1='change var1'
var2='change var2'
}
var1='global 1'
var2='global 2'
echo Before function call: var1 is $var1 : var2 is $var2
var_change
echo After funtion call: var1 is $var1 : var2 is $var2
```

```
rbcs@ -VMware-Virtual-Platform:~$ ./variable.sh
Before function call: var1 is global 1 : var2 is global 2
Inside function: var1 is local1 : var2 is global 2
After funtion call: var1 is global 1 : var2 is change var2
```

ملاحظات:

1. عند تعريف متغير محلي (local) داخل تابع وله نفس اسم المتغير العالمي (global) ، فإن المتغير المستخدم داخل التابع هو المتغير المحلي فقط، لذلك، تبقى قيمة المتغير var1 العالمية دون تغيير بعد استدعاء التابع، لأن جميع التعديلات تمت على النسخة المحلية منه داخل التابع.
2. أما إذا تم استخدام المتغير var1 داخل التابع قبل إعادة تعريفه كمتغير محلي، فسيتم التعامل مع المتغير العالمي في تلك المرحلة، وذلك لأن سكربت الـ Bash يُنفذ سطرًا بسطر.

```
#!/bin/bash
var_change (){
echo var1: $var1
local var1='local1'
echo Inside function: var1 is $var1 : var2 is $var2
var1='change var1'
var2='change var2'
}
var1='global 1'
var2='global 2'
echo Before function call: var1 is $var1 : var2 is $var2
var_change
echo After funtion call: var1 is $var1 : var2 is $var2
~
```

```
rbcs@ -VMware-Virtual-Platform:~$ ./variable.sh
Before function call: var1 is global 1 : var2 is global 2
var1: global 1
Inside function: var1 is local1 : var2 is global 2
After funtion call: var1 is global 1 : var2 is change var2
```

CONDITIONALS IN BASH

تسمح بنى الشرط في Bash باتخاذ قرارات أثناء تنفيذ السكريبت. حيث يتم تقييم شرط معين، وإذا كانت نتيجة التقييم true يتم تنفيذ التعليمة أو مجموعة التعليمات المرتبطة به.

■ الأشكال الأساسية للشرط في Bash

1. البنية الأساسية (Basic if statement):

```
if [expression]; then
    # commands to run if expression is true
fi
```

2. If - else

```
if [expressio]; then
    # true block
else
    # false block
fi
```



3. If - elif - else

```
if [expressio]; then
    #block1
elif [condition2]; then
    #block2
else
    #default block
fi
```

■ معاملات المقارنة العددية (Numeric Comparison Operators in Bash)

المثال	المعنى	Operator
[5 -eq 5] → true	يساوي (Equal to)	-eq
[5 -ne 3] → true	لا يساوي (Not equal to)	-ne
[7 -gt 5] → true	أكبر تماماً (greater than)	-gt
[3 -lt 5] → true	أصغر تماماً (less than)	-lt
[5 -ge 5] → ture	أكبر أو يساوي (greater than or equal)	-ge
[3 -le 5] → true	أصغر أو يساوي (less than or equal)	-le

■ خيارات الاختبار الشائعة (Common test options in Bash)

المثال	المعنى	الخيار
[-z "\$var"] يرجع true إذا قيمة المتغير فارغة	السلسلة النصية فارغة	-z
[-f "/etc/passwd"] يرجع true إذا كان الملف موجوداً	المسار يشير إلى ملف عادي	-f
[-d "/home/user"] يرجع true إذا كان المجلد موجوداً	المسار يشير إلى مجلد	-d

في هذا المثال، نكتب سكربت يأخذ رقمين صحيحين و نوع العملية الحسابية المراد تطبيقها عليهما، ثم يعيد الناتج.

```
#!/bin/bash
# Prompt the user for two integers and an operation
read -p "Enter first integer (a): " a
read -p "Enter second integer (b): " b
read -p "Choose operation (+, -, *, /, %): " op
# Validate that inputs are integers (simple check)
if [[ "$a" =~ ^-?[0-9]+$ && "$b" =~ ^-?[0-9]+$ ]]; then
:
else
    echo "Error: a and b must be integers."
    exit 1
fi
# Perform the arithmetic operation based on the chosen operator
if [[ "$op" == "+" ]]; then
    result=$(( a + b ))
    echo "Result (a + b) = $result"
elif [[ "$op" == "-" ]]; then
    result=$(( a - b ))
    echo "Result (a - b) = $result"
elif [[ "$op" == "*" ]]; then
    result=$(( a * b ))
    echo "Result (a * b) = $result"
elif [[ "$op" == "/" ]]; then
    if [[ "$b" -eq 0 ]]; then
        echo "Error: Division by zero is not allowed."
        exit 2
    fi
    result=$(( a / b ))
    echo "Result (a / b) = $result"
elif [[ "$op" == "%" ]]; then
    if [[ "$b" -eq 0 ]]; then
        echo "Error: Modulo by zero is not allowed."
        exit 3
    fi
    result=$(( a % b ))
    echo "Result (a % b) = $result"
else
    echo "Error: Unknown operation '$op'. Use one of: + - * / %"
    exit 4
fi
# Classify the result using if/elif/else (three-way condition)
if (( result > 0 )); then
    echo "The result is POSITIVE."
elif (( result == 0 )); then
    echo "The result is ZERO."
else
    echo "The result is NEGATIVE."
fi
```


- أمر `read -p`: يقوم بطباعة رسالة للمستخدم وأخذ الإدخال في نفس الوقت، ثم يخزن القيمة في المتغير المحدد بعد الرسالة.
- شرح السطر:

```
if [[ "$a" =~ ^-[0-9]+$ && "$b" =~ ^-[0-9]+$ ]]; then
```

هنا نقوم بالتحقق أن الرقمين المدخلين integers باستعمال تعبير نمطي (regex – regular expression)

- `=~` : تستخدم لمقارنة قيمة متغير مع تعبير نمطي
- شرح التعبير `^[0-9]+$`:
 - `^`: بداية السلسلة
 - `-?`: علامة ناقص اختيارية، لتقبل الأعداد السالبة
 - `[0-9]+`: رقم واحد أو أكثر
 - `$`: نهاية السلسلة
- `&&`: تعني أن الشرط يجب أن يتحقق لكلا الرقمين `$a` و `$b`.
- الرمز `( : )` في Bash يُعرف ب `no-op` أي أنه أمر لا يقوم بأي شيء
- في هذا السياق، يستخدم كحشو داخل `then` لأن Bash يتطلب وجود أمر، حتى لو لم نرغب بتنفيذ أي شيء.
- معنى الاستخدام هنا :
- إذا كان الشرط صحيحاً (أي `$a` و `$b` أعداد صحيحة)، لا يتم تنفيذ أي شيء `(:)` ويستمر السكريبت بعد `fi`.
- إذا لم يتحقق الشرط، تُطبع رسالة خطأ ويتوقف السكريبت `(exit 1)`
- سبب استخدام `[[ ]]` بدلاً من `[ ]`
 - بشكل عام `[[ ]]` هي الشكل الأحدث والأقوى، وأكثر أماناً عند التعامل مع الفراغات أو المتغيرات الفارغة.
 - وفي السطر 7 تحديداً استخدمنا `[[ ]]` لأن `[ ]` لا تدعم التعبيرات النمطية ولا يمكن استخدامها مع `=~`

ملاحظة مهمة: كل شرط `if` يجب أن يُغلق بـ `fi`



أعظم الاكتشافات تبدأ من لحظة فضول واحدة.

مثال على تنفيذ السكريبت:

```
root@master:/home/mohammad/scripts# ./condition-numeric.bash
Enter first integer (a): 6
Enter second integer (b): 2
Choose operation (+, -, *, /, %): +
Result (a + b) = 8
The result is POSITIVE.
root@master:/home/mohammad/scripts# ./condition-numeric.bash
Enter first integer (a): 2
Enter second integer (b): 2
Choose operation (+, -, *, /, %): -
Result (a - b) = 0
The result is ZERO.
root@master:/home/mohammad/scripts# ./condition-numeric.bash
Enter first integer (a): 2
Enter second integer (b): 0
Choose operation (+, -, *, /, %): /
Error: Division by zero is not allowed.
root@master:/home/mohammad/scripts# ./condition-numeric.bash
Enter first integer (a): 2
Enter second integer (b): 0
Choose operation (+, -, *, /, %): %
Error: Modulo by zero is not allowed.
root@master:/home/mohammad/scripts# ./condition-numeric.bash
Enter first integer (a): 2
Enter second integer (b): 1
Choose operation (+, -, *, /, %): #
Error: Unknown operation '#'. Use one of: + - * / %
```



لَا تَبْحَثْ عَنِ الرَّاحَةِ
وَأَنْتَ فِي عُمْرِ السَّعْيِ